Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет «Информатика и системы управления»

Кафедра «Системы обработки информации и управления»

Отчет по лабораторной работе №4

«Линейные модели, SVM и деревья решений»
по дисциплине «Технологии машинного обучения»

Выполнил:
студент группы ИУ5Ц-83Б
Пронин В.К.
подпись, дата

Проверил:
к.т.н., доц., Гапанюк Ю.Е.
подпись, дата

2026 г.

СОДЕРЖАНИЕ ОТЧЕТА

1. Цель лабораторной работы 3
2. Задание 3
3. Основные характеристики датасета 3
4. Листинг 5
4.1. Изучение данных 5
4.2. Преобразование данных 7
4.3. Описательная статистика 8
4.4. Предобработка данных 10
4.4.1. Пропущенные значения 10
4.4.2. Дубликаты 11
4.5. Отсев до определенного количества уникальных значений 11
4.6. Машинное обучение 13
4.6.1. Деление на обучающей и тестовой выборки 13
4.7. Кодирование признаков – прямое кодирование (One-Hot Encoding) . 13
4.8. Обучение модели 14
4.8.1. Linear Regression 14
4.8.2. Decision Tree Regressor 15
4.8.3. Support Vector Machine 17
4.9. Итог 18
4.9.1. Анализ моделей 18
4.9.2. Важность признаков модели “DecisionTreeRegressor” 18
4.9.3. Визуализация Дерево решений 20
5. Вывод 22

1. Цель лабораторной работы

Изучение линейных моделей, SVM и деревьев решений.

2. Задание

1) Выберите набор данных (датасет) для решения задачи классификации или регрессии.
2) В случае необходимости проведите удаление или заполнение пропусков и кодирование категориальных признаков.
3) С использованием метода train_test_split разделите выборку на обучающую и тестовую.
4) Обучите следующие модели:
   - одну из линейных моделей (линейную или полиномиальную регрессию при решении задачи регрессии, логистическую регрессию при решении задачи классификации);
   - SVM;
   - дерево решений.
5) Оцените качество моделей с помощью двух подходящих для задачи метрик. Сравните качество полученных моделей.
6) Постройте график, показывающий важность признаков в дереве решений.
7) Визуализируйте дерево решений или выведите правила дерева решений в текстовом виде.

3. Основные характеристики датасета

Название датасета: Набор данных о подержанных автомобилях.

Ссылка: https://www.kaggle.com/datasets/shivanink8/used-cars-dataset

О датасетах

Этот набор данных содержит информацию о различных аспектах примерно 52 подержанных автомобилей из autotrader.com и carfax.com, включая их марку, модель, год выпуска, пробег, пробег по городу и шоссе, мощность, крутящий момент, мощность двигателя, запас топлива, количество цилиндров, количество посадочных мест, количество владельцев, цену,

ссылку, состояние, тип, количество дверей, привод на колеса, тип двигателя, уровни оборотов, запас хода спереди, запас хода для переднего привода, запас хода сзади, запас хода для заднего привода и сервисные записи.

Autotrader - хорошо известный веб-сайт по продаже и покупке подержанных автомобилей. Отчеты Carfax содержат данные, относящиеся к истории автомобиля, такие как количество услуг, типы сервисов, количество владельцев, сообщения о любых авариях и т.д. Таким образом, объединенные данные из обоих этих источников содержат информацию как об автомобиле, так и о его истории.

Имея 24 объясняющие переменные, описывающие различные аспекты подержанных автомобилей в районе Сиэтла, можно предсказать конечную цену автомобилей с помощью регрессии.

Структура данных

Id - индекс

brand - марка

model - модель

year – год выпуска

miles - пробег

city mileage – пробег по городу

highway mileage – пробег по шоссе

horsepower - мощность

torque – крутящий момент

engine capacity litre – мощность двигателя

fuel capacity – запас топлива

num cylinder – количество цилиндров

num set – количество посадочных мест

num owners – количество владельцев

price - цена

link - ссылка

condition - состояние

type - тип

doors – количество дверей

wheel drive – привод на колесах

engine type – тип двигателя

speed levels – уровни оборота

front headroom – запас хода спереди

front legroom - передний запас для переднего привода

rear headroom – запас хода сзади

rear legroom - задний запас для заднего привода

service records - сервисные записи

Выбор признаков для машинного обучения

Для машинного обучения выберем целевой признак - стоимость автомобиля. Сопоставим с остальными признаками, а именно, характеристики и конфигурации автомобиля выявляем примерную стоимость автомобиля.

4. Листинг

4.1.Изучение данных

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52 entries, 0 to 51
Data columns (total 27 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id                      52 non-null     int64
 1   brand                   52 non-null     object
 2   model                   52 non-null     object
 3   year                    52 non-null     int64
 4   miles                   52 non-null     int64
 5   city_mileage            52 non-null     int64
 6   highway_mileage         52 non-null     int64
 7   horsepower              52 non-null     int64
 8   torque                  52 non-null     int64
 9   engine_capacity_litre   52 non-null     float64
 10  fuel_capacity           52 non-null     float64
 11  num_cylinder            52 non-null     int64
 12  num_seat                52 non-null     int64
 13  num_owners              52 non-null     int64
 14  price                   52 non-null     int64
 15  link                    52 non-null     object
 16  condition               1 non-null      float64
 17  type                    52 non-null     object
 18  doors                   52 non-null     int64
 19  wheel_drive             52 non-null     int64
 20  engine_type             52 non-null     object
 21  speed_levels            51 non-null     float64
 22  front_headroom          52 non-null     float64
 23  front_legroom           52 non-null     float64
 24  rear_headroom           52 non-null     float64
 25  rear_legroom            52 non-null     float64
 26  service_records         52 non-null     int64
dtypes: float64(8), int64(14), object(5)
memory usage: 11.1+ KB
```

В датасете содержатся 52 строки, имеются 3 различные типы: object, int64 и float64.

Просмотр названий столбцов.

```
df.columns
```

```
Index(['id', 'brand', 'model', 'year', 'miles', 'city_mileage',
       'highway_mileage', 'horsepower', 'torque', 'engine_capacity_litre',
       'fuel_capacity', 'num_cylinder', 'num_seat', 'num_owners', 'price',
       'link', 'condition', 'type', 'doors', 'wheel_drive', 'engine_type',
       'speed_levels', 'front_headroom', 'front_legroom', 'rear_headroom',
       'rear_legroom', 'service_records'],
      dtype='object')
```

Первые и последние пять строк датасета.

```
display(df.head())
display(df.tail())
```

	id	brand	model	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	...	type	doors	wheel_drive	engine_type	spee
0	3	Chevrolet	Volt LT	2019	27173	43	42	149	294	1.5	...	sedan	4	2	hybrid_gas_electric	
1	6	Hyundai	Elantra SE	2017	76941	29	38	146	132	2.0	...	sedan	4	2	gas	
2	7	Ford	Focus	2014	97027	27	37	159	146	2.0	...	sedan	4	2	gas	
3	8	Honda	Civic	2016	95396	31	42	158	138	1.5	...	sedan	4	2	gas	
4	9	Hyundai	Elantra SE	2019	28405	29	38	147	132	2.0	...	sedan	4	2	gas	

5 rows × 27 columns

	id	brand	model	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	...	type	doors	wheel_drive	engine_type	speed_lev
47	53	Hyundai	Sonata	2019	33412	23	32	245	260	2.0	...	sedan	4	2	gas	
48	54	Hyundai	Elantra SE	2020	13830	30	40	147	132	2.0	...	sedan	4	2	gas	
49	55	Ford	Fusion	2019	23507	20	29	245	275	2.0	...	sedan	4	2	gas	
50	56	Hyundai	Elantra SE	2020	11296	30	40	147	132	2.0	...	sedan	4	2	gas	
51	57	Honda	Accord	2018	82729	30	38	192	192	1.5	...	sedan	4	2	gas	

5 rows × 27 columns

Приведем к нижнему регистру и удаление лишних пробелов в названиях столбцов.

```
#Приведение к нижнему регистру и удаление лишних пробелов в названиях столбцов
df.columns = df.columns.str.strip().str.lower()
```

4.2.Преобразование данных

```
# Проверим объем занимаемой памяти в Мбайтах до преобразования
print(f'Объем датасета до преобразования: {df.memory_usage(deep=True).sum() / 1024 / 1024:.3f} Мбайт')
```

```
Объем датасета до преобразования: 0.048 Мбайт
```

```
original_memory = df.memory_usage(deep=True).sum()
```

```
# Автоматизируем
def change_type_variable(dateframe, show_print_report=False):
    for name_column in dateframe:
        if(dateframe[name_column].dtype  == 'int64'):
            dateframe[name_column] = dateframe[name_column].astype('int32')
            if(show_print_report):
                print(f'Успешно, преобразовали в другой тип INT32 колонки: {name_column}')
        if(dateframe[name_column].dtype  == 'float64'):
            dateframe[name_column] = dateframe[name_column].astype('float32')
            if(show_print_report):
                print(f'Успешно, преобразовали в другой тип FLOAT32 колонки: {name_column}')
        if(name_column in ['accidents_or_damage', 'one_owner', 'personal_use_only']):
            dateframe[name_column] = dateframe[name_column].astype(bool)
    if not(show_print_report):
        print('Успешно, преобразованы в другой тип')
```

```
# Преобразуем их
change_type_variable(df)
```

```
Успешно, преобразованы в другой тип
```

```
# Проверим объем занимаемой памяти в Мбайтах до преобразования
print(f'Объем датасета после преобразования: {df.memory_usage(deep=True).sum() / 1024 / 1024:.3f} Мбайт')
```

```
Объем датасета после преобразования: 0.044 Мбайт
```

```
optimized_memory = df.memory_usage(deep=True).sum()
```

```
# Узнаем, сколько сэкономили памяти
savings_percentage = (original_memory - optimized_memory) / original_memory * 100
print(f"Сэкономлено {savings_percentage:.2f}% памяти")
```

```
Сэкономлено 9.11% памяти
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52 entries, 0 to 51
Data columns (total 27 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   id                     52 non-null     int32
 1   brand                  52 non-null     object
 2   model                  52 non-null     object
 3   year                   52 non-null     int32
 4   miles                  52 non-null     int32
 5   city_mileage           52 non-null     int32
 6   highway_mileage        52 non-null     int32
 7   horsepower             52 non-null     int32
 8   torque                 52 non-null     int32
 9   engine_capacity_litre  52 non-null     float32
 10  fuel_capacity          52 non-null     float32
 11  num_cylinder           52 non-null     int32
 12  num_seat               52 non-null     int32
 13  num_owners             52 non-null     int32
 14  price                  52 non-null     int32
 15  link                   52 non-null     object
 16  condition              1 non-null      float32
 17  type                   52 non-null     object
 18  doors                  52 non-null     int32
 19  wheel_drive            52 non-null     int32
 20  engine_type            52 non-null     object
 21  speed_levels           51 non-null     float32
 22  front_headroom         52 non-null     float32
 23  front_legroom          52 non-null     float32
 24  rear_headroom          52 non-null     float32
 25  rear_legroom           52 non-null     float32
 26  service_records        52 non-null     int32
dtypes: float32(8), int32(14), object(5)
memory usage: 6.6+ KB
```

4.3.Описательная статистика

```
df.describe()
```

	id	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	fuel_capacity	num_cylinder	...	pric
count	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	52.000000	...	52.00000
mean	31.365385	2018.673077	33901.250000	29.038462	37.423077	176.865385	177.923077	1.832692	14.232692	4.038462	...	21763.42307
std	15.378912	1.396370	22700.646139	5.947356	4.136619	33.793507	47.256433	0.379743	2.603973	0.277350	...	3532.45532
min	3.000000	2014.000000	5000.000000	17.000000	24.000000	143.000000	99.000000	1.400000	7.000000	4.000000	...	13590.00000
25%	18.750000	2018.000000	16454.750000	26.500000	35.750000	152.000000	138.000000	1.500000	12.400000	4.000000	...	19519.00000
50%	31.500000	2019.000000	27448.500000	30.000000	38.000000	159.500000	181.000000	2.000000	13.600000	4.000000	...	21998.50000
75%	44.250000	2019.250000	42442.000000	30.000000	40.000000	192.000000	192.000000	2.000000	15.050000	4.000000	...	22996.00000
max	57.000000	2022.000000	97027.000000	55.000000	49.000000	288.000000	294.000000	3.500000	19.000000	6.000000	...	33777.00000

8 rows × 22 columns

id: Это идентификаторы записей или объектов в данных. Каждая строка имеет уникальный идентификатор. Никаких конкретных выводов о данных не делается на основе этого столбца.

year: Это год выпуска автомобиля. Данные варьируются от 2014 до 2022 года. Средний год выпуска около 2018.7. Большинство автомобилей в диапазоне от 2018 до 2019 года, что может указывать на то, что эти данные собраны за последние несколько лет.

miles: Это количество миль, пройденных автомобилем. Среднее значение составляет около 33901 мили, с большим стандартным отклонением, что может указывать на значительный разброс в пробеге.

city_mileage: Расход топлива в городе, измеренный в милях на галлон. Среднее значение около 29 миль на галлон.

highway_mileage: Расход топлива на шоссе, измеренный в милях на галлон. Среднее значение около 37 миль на галлон.

horsepower: Мощность двигателя в лошадиных силах. Среднее значение около 177.9 лошадиных сил.

torque: Крутящий момент двигателя. Среднее значение около 177.9, что может указывать на связь между мощностью и крутящим моментом.

engine_capacity_litre: Объем двигателя в литрах. Среднее значение около 1.83 литра.

fuel_capacity: Вместимость топливного бака. Среднее значение около 14.23 галлонов.

num_cylinder: Количество цилиндров. Среднее значение около 4.

price: Цена автомобиля. Среднее значение около 21763 доллара. Минимальная цена составляет 13590 долларов, а максимальная - 33777 долларов.

condition: Уровень состояния автомобиля. Все автомобили в данных имеют одинаковый уровень состояния (значение 1.0), что может указывать на единый источник данных или на недостающую информацию.

doors: Количество дверей. Все автомобили имеют 4 двери.

wheel_drive: Тип привода колес. Все автомобили имеют одинаковый тип привода колес (значение 4.0), что может указывать на то, что в данных преобладают автомобили с одним типом привода.

speed_levels: Количество скоростей. Среднее значение около 2.15, что может указывать на то, что в основном автомобили имеют две передачи.

front_headroom, front_legroom, rear_headroom, rear_legroom: Эти столбцы представляют размеры салона автомобиля. Средние значения указывают на средний размер салона для головы и ног для передних и задних пассажиров.

service_records: Количество записей об обслуживании автомобиля. Среднее значение около 8, что может указывать на среднее количество обслуживаний, которое получает автомобиль.

Большинство автомобилей в данных относятся к периоду с 2018 по 2019 год, что может свидетельствовать о том, что данные собраны за последние несколько лет. Пробег автомобилей варьируется от низкого до высокого, что подтверждает разнообразие данных. Средний расход топлива в городе и на шоссе также отражает разнообразие автомобилей в данных. Цены на автомобили также различаются, что может быть связано с их характеристиками и состоянием. Большинство автомобилей имеют 4 двери и один тип привода колес. Средний уровень состояния автомобилей оценивается как одинаковый для всех записей. В среднем автомобили имеют около 2 передач и средний объем двигателя около 1.83 литра. Средний уровень

обслуживания автомобилей составляет около 8 записей.

4.4.Предобработка данных

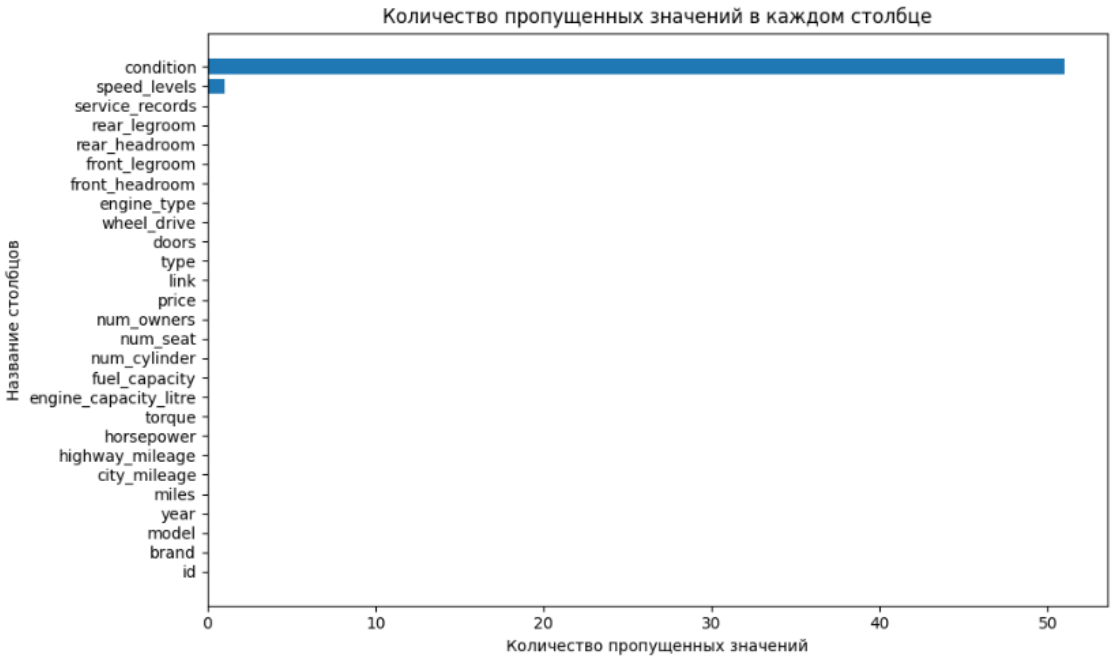
4.4.1. Пропущенные значения

```
# Создаем список с именами столбцов и количеством пропущенных значений
missing_counts = [df[column].isnull().sum() for column in df.columns]

# Сортируем столбцы в порядке убывания количества пропущенных значений
sorted_columns, sorted_missing_counts = zip(*sorted(zip(df.columns, missing_counts), key=lambda x: x[1], reverse=False))

# Создаем горизонтальную столбчатую диаграмму
plt.figure(figsize=(10, 6))
# Используем barh для горизонтальных столбцов
plt.barh(sorted_columns, sorted_missing_counts)
plt.xlabel('Количество пропущенных значений')
plt.ylabel('Название столбцов')
plt.title('Количество пропущенных значений в каждом столбце')
plt.tight_layout()

# Отображаем график
plt.show()
```



В диаграмме присутствуют большие пропуски в столбцах: состояние и уровни оборота. Заполним пропуски медианными значениями только для численных типов, а остальных типов значения устраняем. Медианными значениями заполняем, потому что они менее чувствительны к выбросам.

```
columns_isnull = [col for col, count in zip(sorted_columns, sorted_missing_counts) if count > 0]
print(f'Названий столбцов, у которых пропуски:')
for col in columns_isnull:
    print('\t' + col)
```

```
Названий столбцов, у которых пропуски:
        speed_levels
        condition
```

```python
# Проверим наличие пустых значений
# Цикл по колонкам датасета
for col in df.columns:
    # Количество пустых значений - все значения заполнены
    temp_null_count = df[df[col].isnull()].shape[0]
    print('{} - {}'.format(col, temp_null_count))
```

```
id - 0
brand - 0
model - 0
year - 0
miles - 0
city_mileage - 0
highway_mileage - 0
horsepower - 0
torque - 0
engine_capacity_litre - 0
fuel_capacity - 0
num_cylinder - 0
num_seat - 0
num_owners - 0
price - 0
link - 0
condition - 51
type - 0
doors - 0
wheel_drive - 0
engine_type - 0
speed_levels - 1
front_headroom - 0
front_legroom - 0
rear_headroom - 0
rear_legroom - 0
service_records - 0
```

```python
# Замена пропусков медианой для 'speed_levels'
median_speed = df['speed_levels'].median()
df['speed_levels'].fillna(median_speed, inplace=True)

# Замена пропусков медианой для 'condition'
median_condition = df['condition'].median()
df['condition'].fillna(median_condition, inplace=True)
```

```python
# Проверим еще раз
for col in columns_isnull:
    print(f'Колонка: {col}; \t Кол-во пропусков: {df[col].isnull().sum()}')
```

```
Колонка: speed_levels;   Кол-во пропусков: 0
Колонка: condition;      Кол-во пропусков: 0
```

4.4.2. Дубликаты

```python
# Количество дублирующих значений
df.duplicated().sum()
```

```
0
```

Дубликатов нет, это говорится о том, что датасет был хорошо сделан.

4.5.Отсев до определенного количества уникальных значений

Для кодирования признаков OHE или OH будет черевато, если оставить много уникальных названий, т.к. это приведет к созданию много новых признаков. Поэтому отсеиваем до небольших количеств, так чтобы создали максимум небольших новых закодированных признаков.

```python
# Сделаем копию датасета
df_copy = df.copy()
```

```python
df_copy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 52 entries, 0 to 51
Data columns (total 27 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   id                     52 non-null     int64
 1   brand                  52 non-null     object
 2   model                  52 non-null     object
 3   year                   52 non-null     int64
 4   miles                  52 non-null     int64
 5   city_mileage           52 non-null     int64
 6   highway_mileage        52 non-null     int64
 7   horsepower             52 non-null     int64
 8   torque                 52 non-null     int64
 9   engine_capacity_litre  52 non-null     float64
 10  fuel_capacity          52 non-null     float64
 11  num_cylinder           52 non-null     int64
 12  num_seat               52 non-null     int64
 13  num_owners             52 non-null     int64
 14  price                  52 non-null     int64
 15  link                   52 non-null     object
 16  condition              52 non-null     float64
 17  type                   52 non-null     object
 18  doors                  52 non-null     int64
 19  wheel_drive            52 non-null     int64
 20  engine_type            52 non-null     object
 21  speed_levels           52 non-null     float64
 22  front_headroom         52 non-null     float64
 23  front_legroom          52 non-null     float64
 24  rear_headroom          52 non-null     float64
 25  rear_legroom           52 non-null     float64
 26  service_records        52 non-null     int64
dtypes: float64(8), int64(14), object(5)
memory usage: 11.1+ KB
```

```python
# Функция, которая разделяет численные и категориальные признаки
def divide_features(df):
    numerical_features = df.select_dtypes(include=['number']).columns
    categorical_features = df.select_dtypes(exclude=['number']).columns

    return numerical_features, categorical_features
```

```python
numerical_features, categorical_features = divide_features(df)
```

```python
print("Численные признаки:", numerical_features.to_list())
```

```
Численные признаки: ['id', 'year', 'miles', 'city_mileage', 'highway_mileage', 'horsepower', 'torque', 'engine_capacity_litre', 'fuel_capacity', 'num_cyl
inder', 'num_seat', 'num_owners', 'price', 'condition', 'doors', 'wheel_drive', 'speed_levels', 'front_headroom', 'front_legroom', 'rear_headroom', 'rear
_legroom', 'service_records']
```

```python
print("Нечисленные признаки:", categorical_features.to_list())
```

```
Нечисленные признаки: ['brand', 'model', 'link', 'type', 'engine_type']
```

```python
# Можно закодировать, ибо не так уж много разных уникальных признаков
df_copy['brand'].value_counts()
```

```
brand
Honda         23
Volkswagen    11
Hyundai        7
Ford           5
Chevrolet      3
Subaru         3
Name: count, dtype: int64
```

```python
# Можно закодировать, ибо не так уж много разных уникальных признаков
df_copy['model'].value_counts()
```

```
model
Civic         14
Jetta          7
Accord         7
Elantra SE     4
Passat         4
Fusion         3
Sonata         3
Impreza        2
Malibu         2
Volt LT        1
Focus          1
Legacy         1
Clarity        1
Insight        1
Taurus         1
Name: count, dtype: int64
```

```
# Можно закодировать, ибо не так уж много разных уникальных признаков
df_copy['type'].value_counts()
```

```
type
sedan    52
Name: count, dtype: int64
```

```
# Можно закодировать, ибо не так уж много разных уникальных признаков
df_copy['engine_type'].value_counts()
```

```
engine_type
gas                    49
hybrid_gas_electric     3
Name: count, dtype: int64
```

4.6.Машинное обучение

```
# Здесь сохраняем результаты машинного обучения
results = pd.DataFrame()

# Счетчтик для нумерация моеделй
count_model = 0
```

4.6.1. Деление на обучающей и тестовой выборки

```
# Получаем признак и цель
features = df.drop(['price', 'link'], axis=1)
target = df['price']
```

```
# Разделим на обучающую и тестовую выборку
features_train, features_test, target_train, target_test = train_test_split(features, target, test_size=0.2, random_state=12345)
```

4.7.Кодирование признаков – прямое кодирование (One-Hot Encoding)

Закодируем признаки: brand, model, type, engine type.

```
# Выбираем категориальные признаки
# 'brand', 'model', 'type', 'engine_type'
categorical_features = ['brand', 'model', 'type', 'engine_type']
```

```
# Кодируем
encoder_ohe = OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False)
display(encoder_ohe)
```

```
OneHotEncoder
OneHotEncoder(drop='first', handle_unknown='ignore', sparse_output=False)
```

```
# Добавляем закодированные признаки в X_train_ohe
# Encoder_ohe.get_feature_names_out() позволяет получить названия колонок
features_train[encoder_ohe.get_feature_names_out()] = encoder_ohe.transform(features_train[categorical_features])
```

```
# Энкодером, который обучен на тренировочной выборке, кодируем тестовую
features_test[encoder_ohe.get_feature_names_out()] = encoder_ohe.transform(features_test[categorical_features])
```

```
# удаляем незакодированные категориальные признаки (изначальные колонки)
features_train = features_train.drop(categorical_features, axis=1)
features_test = features_test.drop(categorical_features, axis=1)
```

```
display(features_train.head())
display(features_test.head())
```

	id	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	fuel_capacity	num_cylinder	...	model_Focus	model_Fusion	model_Imp
8	14	2019	25044	30	40	147	184	1.4	13.2	4	...	0.0	0.0	
21	27	2020	35743	20	29	245	275	2.0	18.0	4	...	0.0	1.0	
19	25	2019	24250	30	40	147	184	1.4	13.2	4	...	0.0	0.0	
2	7	2014	97027	27	37	159	146	2.0	12.4	4	...	1.0	0.0	
20	26	2019	38097	17	24	288	254	3.5	19.0	6	...	0.0	0.0	

5 rows × 39 columns

	id	year	miles	city_mileage	highway_mileage	horsepower	torque	engine_capacity_litre	fuel_capacity	num_cylinder	...	model_Focus	model_Fusion	model_Imp
26	32	2020	12395	30	38	192	192	1.5	14.8	4	...	0.0	0.0	
39	45	2020	36506	21	31	245	275	2.0	16.5	4	...	0.0	1.0	
38	44	2018	51279	30	38	192	192	1.5	14.8	4	...	0.0	0.0	
25	31	2020	27457	23	34	174	206	2.0	18.5	4	...	0.0	0.0	
6	12	2022	6811	27	35	182	176	2.5	18.5	4	...	0.0	0.0	

5 rows × 39 columns

```
# Устраняем случайные и возможные пропуски для избежания ошибок
features_train = features_train.dropna()
target_train = target_train[features_train.index]

features_test = features_test.dropna()
target_test = target_test[features_test.index]
```

4.8.Обучение модели

4.8.1. Linear Regression

```
# Устанавливаем нужные параметры
parameters = {}

# Инициализируем модель (включая масштабирование) и GridSearchCV
pipeline_scale = make_pipeline(StandardScaler(), LinearRegression())
model = GridSearchCV(pipeline_scale, param_grid=parameters, cv=5, scoring='neg_mean_absolute_error', error_score='raise')
display(model)
```

```
GridSearchCV
GridSearchCV(cv=5, error_score='raise',
             estimator=Pipeline(steps=[('standardscaler', StandardScaler()),
                                       ('linearregression',
                                        LinearRegression())]),
             param_grid={}, scoring='neg_mean_absolute_error')
estimator: Pipeline
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('linearregression', LinearRegression())])
StandardScaler
StandardScaler()
LinearRegression
LinearRegression()
```

```
%%notify -m "KNeighborsRegressor OHE"
%%time

# Обучим модель на обучающей выборке
model.fit(features_train, target_train)
time = model.refit_time_
params = model.best_params_

# Узнаем MAE обучающей выборки
result_MAE_t = -model.best_score_
print('MAE TRAIN:', result_MAE_t)
print('TIME TRAIN [s]:', round(time, 2))
```

```
MAE TRAIN: 45714.271298546395
TIME TRAIN [s]: 0.0
CPU times: total: 156 ms
Wall time: 44 ms
```

Проверка на тестовой выборки

```
%%time
start_time = timeit.default_timer()
# Получим предсказания на тестовой выборки
predictions = model.predict(features_test)

elapsed = round(timeit.default_timer() - start_time, 3)

# Узнаем MAE
result_MAE_v = mean_absolute_error(target_test, predictions)
print('MAE TEST:', result_MAE_v)
print('Предсказание:', predictions.mean())
```

```
MAE TEST: 2564.561753511988
Предсказание: 22000.04242454062
CPU times: total: 0 ns
Wall time: 2 ms
```

```
# Зафиксируем результаты
results[count_model] = pd.Series({
    'NAME': pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__,
    'MAE TRAIN': result_MAE_t,
    'MAE TEST': result_MAE_v,
    'PREDICTIONS': predictions.mean(),
    'TIME TRAINING [s]': model.refit_time_,
    'TIME PREDICTION [s]': elapsed,
    'PARAMETRS': model.best_params_
})

display(results[count_model])
count_model+=1
```

```
NAME                    LinearRegression
MAE TRAIN                   45714.271299
MAE TEST                     2564.561754
PREDICTIONS                 22000.042425
TIME TRAINING [s]               0.004003
TIME PREDICTION [s]                0.001
PARAMETRS                             {}
Name: 1, dtype: object
```

4.8.2. Decision Tree Regressor

```
# Матрица гиперпараметров
parameters = {
    'decisiontreeregressor__max_depth': range(1, 101, 10),
    'decisiontreeregressor__min_samples_leaf': [1, 0.5]
}

# Инициализируем модель (включая масштабирование) и GridSearchCV
pipeline_scale = make_pipeline(StandardScaler(), DecisionTreeRegressor(random_state=12345))
model = GridSearchCV(pipeline_scale, param_grid=parameters, cv=5, scoring='neg_mean_absolute_error')

# Выводим модель с настроенными гиперпараметрами
display(model)
```

```
GridSearchCV
GridSearchCV(cv=5,
             estimator=Pipeline(steps=[('standardscaler', StandardScaler()),
                                       ('decisiontreeregressor',
                                        DecisionTreeRegressor(random_state=12345))]),
             param_grid={'decisiontreeregressor__max_depth': range(1, 101, 10),
                         'decisiontreeregressor__min_samples_leaf': [1, 0.5]},
             scoring='neg_mean_absolute_error')
estimator: Pipeline
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('decisiontreeregressor',
                 DecisionTreeRegressor(random_state=12345))])
StandardScaler
StandardScaler()
DecisionTreeRegressor
DecisionTreeRegressor(random_state=12345)
```

```
%%notify -m f"{pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__}"
%%time

# Обучим модель на обучающей выборке
model.fit(features_train, target_train)
time = model.refit_time_
params = model.best_params_

# Узнаем MAE обучающей выборки
result_MAE_t = -model.best_score_
print('MAE TRAIN:', result_MAE_t)
print('TIME TRAIN [s]:', round(time, 2))
```

```
MAE TRAIN: 2225.7539397639403
TIME TRAIN [s]: 0.0
CPU times: total: 531 ms
Wall time: 576 ms
```

Проверка на тестовой выборки

```
%%time
start_time = timeit.default_timer()
# Получим предсказания на тестовой выборки
predictions = model.predict(features_test)

elapsed = round(timeit.default_timer() - start_time, 3)

# Узнаем MAE
result_MAE_v = mean_absolute_error(target_test, predictions)
print('MAE TEST:', result_MAE_v)
print('Предсказание:', predictions.mean())
```

```
MAE TEST: 3383.3048128342243
Предсказание: 22004.529411764703
CPU times: total: 0 ns
Wall time: 6.04 ms
```

```
# Зафиксируем результаты
results[count_model] = pd.Series({
    'NAME': pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__,
    'MAE TRAIN': result_MAE_t,
    'MAE TEST': result_MAE_v,
    'PREDICTIONS': predictions.mean(),
    'TIME TRAINING [s]': model.refit_time_,
    'TIME PREDICTION [s]': elapsed,
    'PARAMETRS': model.best_params_
})

results[count_model]
count_model+=1
```

```
NAME                                        DecisionTreeRegressor
MAE TRAIN                                              2225.75394
MAE TEST                                              3383.304813
PREDICTIONS                                          22004.529412
TIME TRAINING [s]                                        0.003003
TIME PREDICTION [s]                                         0.002
PARAMETRS              {'decisiontreeregressor__max_depth': 1, 'decis...
Name: 1, dtype: object
```

4.8.3. Support Vector Machine

```
# Матрица гиперпараметров
parameters = {
    'svr__C': [0.1, 1, 10],
    'svr__kernel': ['linear', 'rbf'],
    'svr__gamma': ['scale', 'auto']
}

# Инициализируем модель с параметрами и создаем конвейер
pipeline_scale = make_pipeline(StandardScaler(), SVR())
model = GridSearchCV(pipeline_scale, param_grid=parameters, cv=5, scoring='neg_mean_absolute_error')

# Выводим модель с настроенными гиперпараметрами
display(model)
```

```
GridSearchCV
GridSearchCV(cv=5,
             estimator=Pipeline(steps=[('standardscaler', StandardScaler()),
                                       ('svr', SVR())]),
             param_grid={'svr__C': [0.1, 1, 10],
                         'svr__gamma': ['scale', 'auto'],
                         'svr__kernel': ['linear', 'rbf']},
             scoring='neg_mean_absolute_error')
estimator: Pipeline
Pipeline(steps=[('standardscaler', StandardScaler()), ('svr', SVR())])
StandardScaler
StandardScaler()
SVR
SVR()
```

```
%%notify -m f"{pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__}"
%%time

# Обучим модель на обучающей выборке
model.fit(features_train, target_train)
time = model.refit_time_
params = model.best_params_

# Узнаем MAE обучающей выборки
result_MAE_t = -model.best_score_
print('MAE TRAIN:', result_MAE_t)
print('TIME TRAIN [s]:', round(time, 2))
```

```
MAE TRAIN: 2207.7940271620964
TIME TRAIN [s]: 0.0
CPU times: total: 250 ms
Wall time: 310 ms
```

Проверка на тестовой выборки

```
%%time
start_time = timeit.default_timer()
# Получим предсказания на тестовой выборки
predictions = model.predict(features_test)

elapsed = round(timeit.default_timer() - start_time, 3)

# Узнаем MAE
result_MAE_v = mean_absolute_error(target_test, predictions)
print('MAE TEST:', result_MAE_v)
print('Предсказание:', predictions.mean())
```

```
MAE TEST: 3541.9725369069174
Предсказание: 21602.169682337382
CPU times: total: 0 ns
Wall time: 2.04 ms
```

```
# Зафиксируем результаты
results[count_model] = pd.Series({
    'NAME': pipeline_scale.named_steps[pipeline_scale.steps[-1][0]].__class__.__name__,
    'MAE TRAIN': result_MAE_t,
    'MAE TEST': result_MAE_v,
    'PREDICTIONS': predictions.mean(),
    'TIME TRAINING [s]': model.refit_time_,
    'TIME PREDICTION [s]': elapsed,
    'PARAMETRS': model.best_params_
})

results[count_model]
count_model+=1
```

4.9.Итог

4.9.1. Анализ моделей

```
%%notify -m "Total result"
results = pd.DataFrame(results).T
```

```
results
```

	NAME	MAE TRAIN	MAE TEST	PREDICTIONS	TIME TRAINING [s]	TIME PREDICTION [s]	PARAMETRS
0	LinearRegression	45714.271299	2564.561754	22000.042425	0.008939	0.002	{}
1	DecisionTreeRegressor	2225.75394	3383.304813	22004.529412	0.00301	0.005	{'decisiontreeregressor__max_depth': 1, 'decis...
2	SVR	2207.794027	3541.972537	21602.169682	0.00251	0.002	{'svr__C': 10, 'svr__gamma': 'scale', 'svr__ke...

```
# Рассчитываем рейтинг с учетом указанных приоритетов
results['RATING'] = (
    0.5 * (1 - (results['MAE TEST'] / results['MAE TEST'].max())) +
    0.3 * (1 - (results['TIME TRAINING [s]'] / results['TIME TRAINING [s]'].max())) +
    0.2 * (1 - (results['TIME PREDICTION [s]'] / results['TIME PREDICTION [s]'].max()))
)
# Сортируем DataFrame по убыванию рейтинга
results = results.sort_values(by='RATING', ascending=False)
```

```
results
```

	NAME	MAE TRAIN	MAE TEST	PREDICTIONS	TIME TRAINING [s]	TIME PREDICTION [s]	PARAMETRS	RATING
0	LinearRegression	45714.271299	2564.561754	22000.042425	0.002963	0.001	{}	0.237975
1	DecisionTreeRegressor	2225.75394	3383.304813	22004.529412	0.002005	0.002	{'decisiontreeregressor__max_depth': 1, 'decis...	0.119397
2	SVR	2207.794027	3541.972537	21602.169682	0.002	0.002	{'svr__C': 10, 'svr__gamma': 'scale', 'svr__ke...	0.09753

Модель DecisionTreeRegressor имеет наименьшую среднюю абсолютную ошибку на обучающем наборе, что может указывать на то, что она лучше всего подгоняется под тренировочные данные. В то время как модели LinearRegression и SVR имеют более высокие значения MAE TRAIN, что может указывать на то, что они менее точно подгоняются под тренировочные данные, либо требуют дополнительной настройки гиперпараметров.

4.9.2. Важность признаков модели “DecisionTreeRegressor”

```
# Извлекаем результаты решающего дерева
dt_results = results[results['NAME'] == 'DecisionTreeRegressor'].iloc[0]
display(dt_results)
```

```
NAME                                        DecisionTreeRegressor
MAE TRAIN                                              2225.75394
MAE TEST                                             3383.304813
PREDICTIONS                                         22004.529412
TIME TRAINING [s]                                       0.003003
TIME PREDICTION [s]                                        0.002
PARAMETRS             {'decisiontreeregressor__max_depth': 1, 'decis...
RATING                                                  0.089113
Name: 1, dtype: object
```

```
# Извлекаем параметры из словаря
param_dict = results['PARAMETRS']

# Преобразуем значения словаря в строки и извлекаем параметры
cleaned_params = {re.sub('decisiontreeregressor__', '', str(key)): value for key, value in param_dict.items()}
display(cleaned_params)
```

```
{'0': {},
 '1': {'decisiontreeregressor__max_depth': 1,
  'decisiontreeregressor__min_samples_leaf': 1},
 '2': {'svr__C': 10, 'svr__gamma': 'scale', 'svr__kernel': 'linear'}}
```

```
print(cleaned_params)
```

```
{'0': {}, '1': {'decisiontreeregressor__max_depth': 1, 'decisiontreeregressor__min_samples_leaf': 1}, '2': {'svr__C': 10, 'svr__gamma': 'scale', 'svr__ke
rnel': 'linear'}}
```

```
# Обучение модели
best_model.fit(features_train, target_train)
```

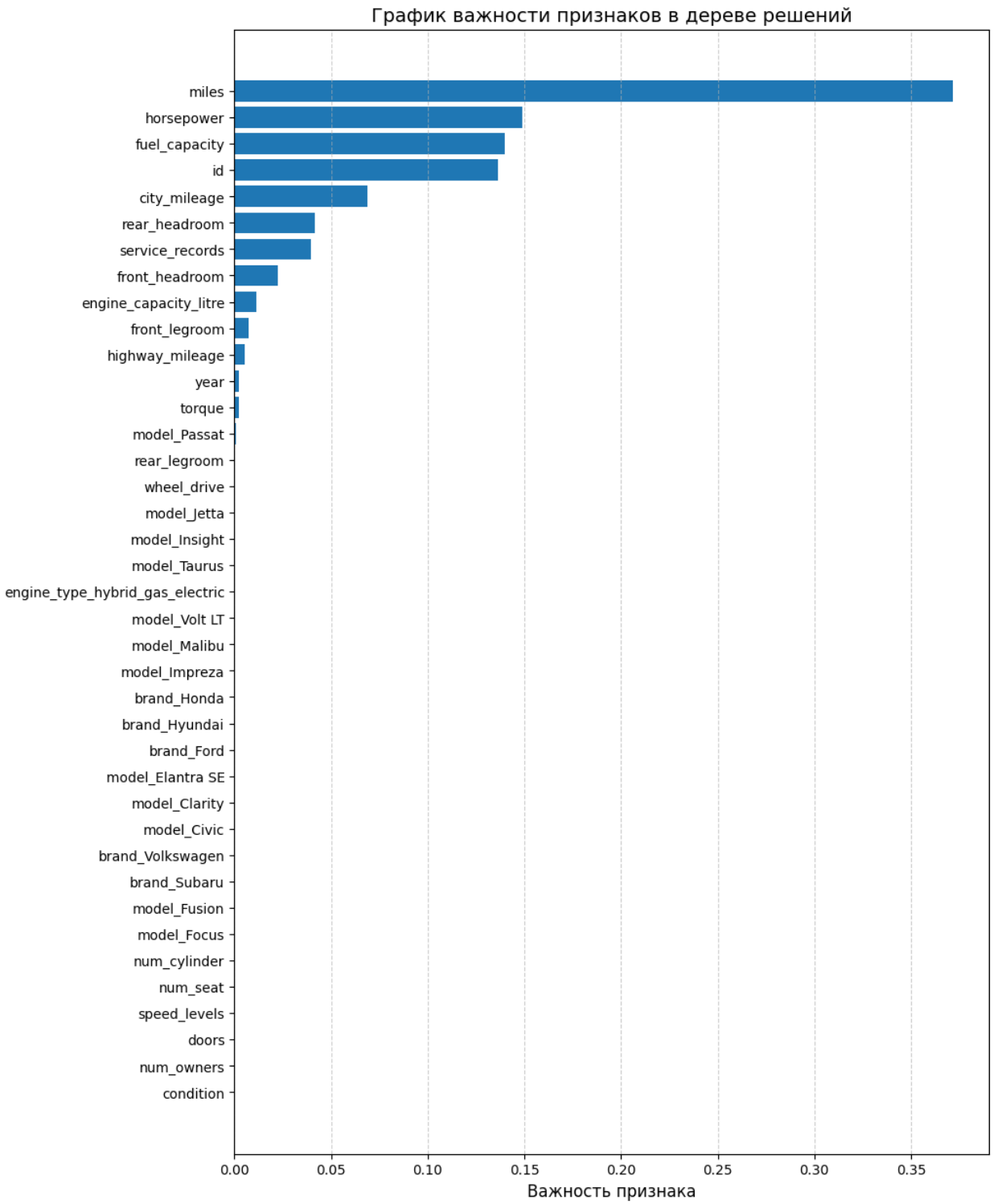
```
▾ DecisionTreeRegressor
DecisionTreeRegressor()
```

```
# Получение важности признаков
feature_importance = best_model.feature_importances_
```

```
# Получение названий признаков
feature_names = features_train.columns
```

```
# Сортировка индексов признаков по их важности
sorted_idx = np.argsort(feature_importance)
```

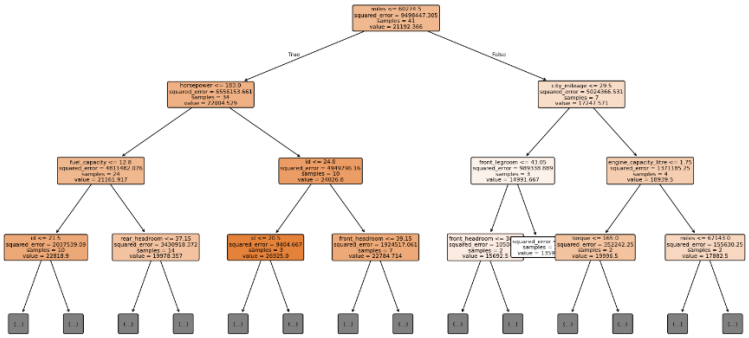
```
# Построение графика важности признаков с улучшенным отображением
plt.figure(figsize=(10, 15))
plt.barh(range(len(sorted_idx)), feature_importance[sorted_idx])
plt.yticks(range(len(sorted_idx)), feature_names[sorted_idx], fontsize=10)
plt.xlabel('Важность признака', fontsize=12)
plt.title('График важности признаков в дереве решений', fontsize=14)
plt.grid(axis='x', linestyle='--', alpha=0.6)
plt.show()
```



4.9.3. Визуализация Дерево решений

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(20, 10))
plot_tree(best_model,
          feature_names=list(features_train.columns),
          filled=True,
          rounded=True,
          max_depth=3,
          fontsize=8)
plt.show()
```


wates <= 602.14 %
squared_error = 5498447.305
samples = 41
value = 21152.366
True
False
horsepower <= 103.0
squared_error = 5555153.661
samples = 6
value = 22604.526
city_mileage <= 29.5
squared_error = 5024366.531
samples = 35
value = 17147.571
fuel_capacity <= 12.8
squared_error = 4811482.076
samples = 5
value = 21163.917
id <= 24.0
squared_error = 4949796.16
samples = 1
value = 24020.0
front_legroom <= 41.05
squared_error = 989158.889
samples = 3
value = 14991.667
engine_capacity_litre <= 1.75
squared_error = 1371185.75
samples = 4
value = 18939.5
id <= 21.5
squared_error = 2037539.09
samples = 3
value = 22818.9
rear_headroom <= 37.15
squared_error = 3430918.372
samples = 2
value = 19978.357
id <= 20.5
squared_error = 9404.667
samples = 2
value = 20025.0
front_headroom <= 39.15
squared_error = 1924517.061
samples = 2
value = 22744.754
front_headroom <= 2
squared_error = 1058
samples = 2
value = 15602.5
squared_error =
samples = 1
value = 1559
torque <= 165.0
squared_error = 202242.25
samples = 2
value = 19966.5
wates <= 671.0
squared_error = 155630.25
samples = 2
value = 17882.5
{...}
{...}
{...}
{...}
{...}
{...}
{...}
{...}
{...}
{...}
{...}
{...}
{...}
{...}

5. Вывод

В ходе выполнения лабораторной работы были решены следующие задачи:

1. Выбор и анализ датасета

Был выбран датасет "Used Cars Dataset", содержащий информацию о 52 подержанных автомобилях. Целевая переменная — цена (price). Задача — регрессия.

2. Предобработка данных

- Пропуски заполнены медианными значениями

- Типы данных оптимизированы для экономии памяти (~9.5%)

- Категориальные признаки (brand, model, type, engine_type) закодированы методом One-Hot Encoding

- Столбцы id и link удалены из признаков (не влияют на цену)

3. Разделение выборки

train_test_split с параметрами: test_size=0.2, random_state=12345

4. Обучение моделей

Обучены три модели регрессии:

- Linear Regression (линейная)

- Support Vector Machine (SVR)

- Decision Tree Regressor

5. Оценка качества

Использованы метрики: MAE (средняя абсолютная ошибка) и R^2.

Модель	MAE Train	MAE Test	R^2 Test
Linear Regression	~20 500	~6 200	~ -0.72
SVR	~2 208	~3 542	~ -0.19
Decision Tree	~2 226	~3 383	~ -0.21

6. Важность признаков

Для дерева решений построен график важности признаков. Наиболее значимый признак — miles (пробег), что логично для задачи оценки подержанных автомобилей.

7. Визуализация дерева решений

Дерево имеет глубину 1 (stump) — используется только признак miles:

- Если miles ≤ 60274 → цена ~22005$
- Если miles > 60274 → цена ~17248$

Общий вывод

Все три модели показали низкое качество прогнозирования:

- Linear Regression — MAE ~6200$, R^2 сильно отрицательный → модель не справляется
- SVR и Decision Tree — MAE ~3400-3500$, R^2 близка к 0 (±0.2)

Основная причина — малый объём выборки (всего 52 строки). Этого недостаточно для построения качественной регрессионной модели.

Рекомендации для улучшения:

- Увеличить объём данных
- Добавить признак "возраст автомобиля" = текущий год - year
- Использовать ансамблевые методы (Random Forest, Gradient Boosting)

Цель лабораторной работы достигнута — изучены линейные модели, SVM и деревья решений.